

지식그래프를 AstraDB에 NoSQL로 담는 법

그래프 DB가 아니라 문서형 NoSQL(Data API) 컬렉션 하나에 엔티티와 관계를 어떻게 저장하고, 어떻게 다시 그래프로 되살려 쓰는지 — 라이브 데이터로 설명합니다.

DataStax AstraDB · IBM watsonx.data

JSON Data API

단일 컬렉션 + kind 필드

임베딩 시드 → 1-홉

2

컬렉션
kg · doc_registry

100

엔티티(노드)
kind = "entity" (+emb)

108

엣지(관계)
kind = "edge"

8

적재 문서
md · PDF · URL

```
# Astra keyspace: default_keyspace
default_keyspace
├─ kg // 지식그래프 = 엔티티 + 엣지 (한 컬렉션에 같이)
│   ├── {kind:"entity"} × 100 노드 (+ emb: 768-d 임베딩)
│   └─ {kind:"edge"} × 108 관계(트리플)
└─ doc_registry // 적재 문서 추적 × 8 (md·PDF·URL)
```

핵심: 테이블도 그래프 엔진도 없습니다. 노드와 엣지를 같은 컬렉션에 넣고 kind 필드 하나로 구분합니다. 그래프 순회(Cypher/Gremlin)는 쓰지 않고, 질의 때 앱이 메모리에서 1-홉을 구성합니다.

한 컬렉션, 두 종류의 문서

모든 KG 데이터는 **평범한 JSON 문서**입니다. **kind** 값이 **entity**면 노드, **edge**면 관계(트리플)입니다. 둘 다 **doc_id**로 "어느 원문에서 나왔는지"를 추적하고, **norm** 계열 필드로 표기 흔들림을 흡수합니다.

kind: "entity" 엔티티 — 노드

개념·기관·기술 등 "무엇"

kind: "edge" 엣지 — 관계 트리플

(출발)—관계→(도착) "어떻게 연결"

A 엔티티 문서 (실데이터)

```
{
  "kind": "entity",
  "name": "이상거래탐지시스템(FDS)", // 원문 표기 그대로 (표시용)
  "norm": "이상거래탐지시스템", // 정규화 키 - 매칭·중복병합용
  "type": "system", // 통제 온톨로지 (law.regulator.system...)
  "emb": [0.013, -0.021, ...], // granite 768-d 임베딩 - 시맨틱 시드·해소용
  "doc_id": "98b3454c..." // 출처 문서 (registry와 연결)
}
```

B 엣지 문서 — 트리플 (실데이터)

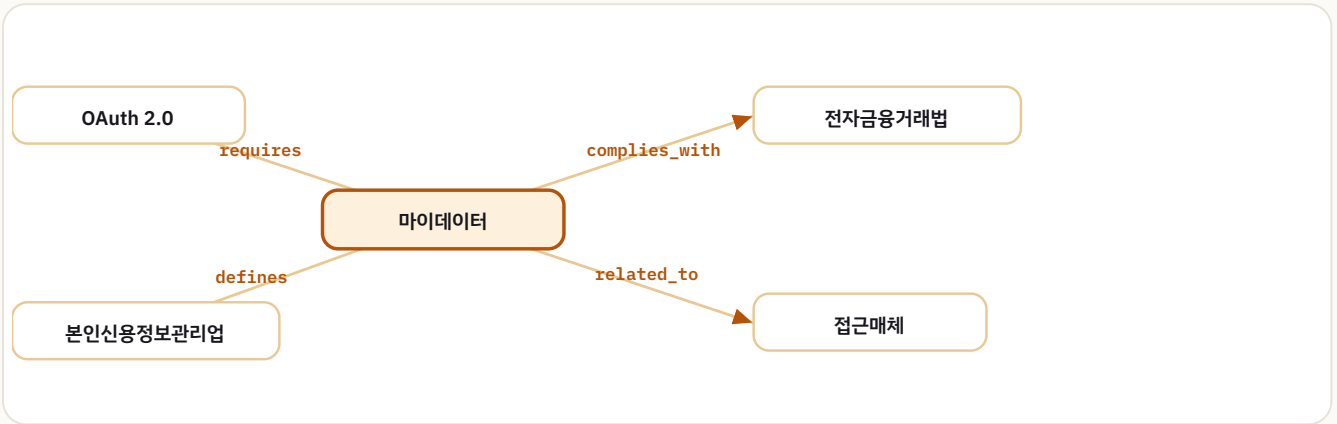
```
{
  "kind": "edge",
  "src": "본인신용정보관리업", "rel": "complies_with", "dst": "신용정보법",
  //      └─ 출발 노드 ─────────── 관계 라벨 ─────────── 도착 노드
  "src_norm": "마이데이터", "dst_norm": "신용정보법", // 별칭→캐논 (본인신용정보관리업→마이데이터)
  "doc_id": "51582e40..."
}
```

→ 읽으면 "마이데이터(=본인신용정보관리업)는 신용정보법을 준수한다". 이런 행(=문서)이 108개 나열된 **엣지 리스트**가 곧 그래프입니다.

왜 **norm**과 **emb**이 있나? "마이데이터"와 "마이데이터 사업자", "FDS"와 "이상거래탐지시스템(FDS)"를 **한 노드로** 묶기 위해서입니다. 2단 엔티티 해소 — ① 정규화 키(**norm**)로 표기 흔들림 흡수, ② 임베딩(**emb**) 코사인으로 의미가 같은 별칭 병합. 화면엔 원문 표기(**name**)를 보여줍니다.

실제로 들어있는 그래프

아래는 운영 중인 AstraDB에서 그대로 가져온 트리플입니다. 평평한 엣지 리스트지만, `src_norm`으로 모으면 이렇게 연결됩니다.



엣지 리스트 (원본)

신용정보법 **regulates**→ 마이데이터
 마이데이터 **related_to**→ 접근매체
 마이데이터 **requires**→ OAuth 2.0
 본인신용정보관리업 **supervised_by**→ 금융위원회
 의심거래보고(STR) **complies_with**→ 특정금융정보법
 마이데이터 사업자 **reports_to**→ 금융감독원

관계 라벨 분포

regulates		13
requires		12
applies_to		11
complies_with		10
defines		6
supervised_by		5
+ 27 종		각 1~

관계 라벨(**rel**)은 LLM이 자유 추출하되 **snake_case** 통제 어휘로 정규화합니다(한글 라벨 → 영문 표준화). 그래서 같은 의미의 관계가 제각각 흩어지지 않고 위처럼 집계됩니다.

왜 이렇게 했고, 어떻게 동작하나

설계 선택 3가지

1 그래프 DB를 쓰지 않는다

AstraDB Data API(문서형 NoSQL)에 JSON으로 저장. Gremlin·Cypher 같은 그래프 순회 엔진이 없습니다. 운영 부담이 적고, RAG에 필요한 건 작은 1-홉 이웃이라 충분합니다.

2 임베딩 시드 → 1-홉 이웃 조회

질문을 임베딩해 엔티티 `emb` 와 코사인으로 의미적 시드를 고르고, 그 1-홉 이웃 엣지만 `find($or/$in)` 로 조회합니다. 이 AstraDB는 서버사이드 벡터 ANN을 지원하지 않아 코사인은 앱-사이드로 계산(KG 100건 규모라 충분).

3 정규화 + 임베딩으로 노드를 병합한다

`norm` · `src_norm` · `dst_norm` 이 표기 흔들림을 흡수하고, 임베딩 코사인이 의미가 같은 별칭을 추가 병합합니다(2단 해소). 엔티티 `type` 은 통제 온톨로지로 고정합니다.

→ graph 질의가 답이 되기까지

1 도구 선택 — 에이전트(Langflow · granite)가 관계형 질문에 `lookup_relationships` (graph) 도구를 호출

2 벡터 시드 — 질문 임베딩 ↔ 엔티티 `emb` 코사인으로 의미적으로 가까운 시드 엔티티 선정

3 1-홉 이웃 조회 — 시드에 연결된 엣지만 `find($or:[src_normES, dst_normES])` 로 로드, 질문 언급에 가점

4 정리 — 정규화 키로 중복·자기참조 제거 → 엔티티 칩 + 엣지 트리플

5 합성 — 이 서브그래프를 컨텍스트로 답변 생성, UI는 같은 서브그래프를 근거 패널로 노출

요약 — AstraDB에는 "노드(+임베딩)와 엣지를 적은 JSON 문서"가 한 컬렉션에 평평하게 들어있을 뿐입니다. 그래프의 "연결"은 저장기 아니라 질의 시점에 의미 임베딩으로 시드를 찾고 정규화 키로 이어 붙여 만들어집니다. 작은 도메인 KG에 맞는, 단순하고 설명 가능한 설계입니다.